Compiler-Aware QuantumDARTS: Differentiable Quantum Architecture Search for Max-Cut with Compiler-Based Optimization

Rajath Shashikanth Vasisth
University of Colorado Boulder
Advisor: Prof. Ramin Ayanzadeh

Abstract

In this report, we reproduce and extend QuantumDARTS, a method for automatically designing quantum circuits, by studying the effect of compiler awareness on solving the Max-Cut optimization problem. We implement two circuit design strategies from the original work: a global search that optimizes the entire circuit and a local search that learns a small circuit block reused across the graph. We then introduce a compiler-aware extension that accounts for practical hardware constraints by incorporating circuit size, depth, required SWAP operations, and estimated error rates from an IBM quantum backend into the training objective. Experiments on graphs with 6 and 8 nodes compare the original local-search method with the compiler-aware version. We evaluate solution quality using CVaR, along with circuit complexity and noisy simulation metrics. While compiler-aware training alters the resulting circuit structures and reduces compilation overhead, performance under realistic noise remains limited by hardware error rates and circuit depth.

1 Introduction

Variational quantum algorithms (VQAs) are a leading paradigm for near-term quantum computing, particularly for solving optimization problems on noisy intermediate-scale quantum (NISQ) devices. Among these problems, *Max-Cut* is a widely used benchmark due to its simple formulation and relevance to graph optimization. Given an undirected graph, the objective is to partition the vertices into two sets such that the number of edges crossing the partition is maximized. Despite its simplicity, achieving high-quality solutions using quantum circuits remains challenging due to circuit expressivity, optimization difficulty, and hardware noise.

A key limitation of many VQAs is the reliance on manually designed quantum circuit ansätze. Fixed circuit templates may not be well suited to a given problem instance or hardware platform, often leading to unnecessary circuit depth and degraded performance under noise. This has motivated increasing interest in *quantum architecture search* (QAS), which aims to automatically discover effective quantum circuit structures using algorithmic search techniques rather than human design.

QuantumDARTS is a recently proposed differentiable quantum architecture search framework that adapts ideas from classical neural architecture search to the quantum setting. By relaxing discrete gate choices into a continuous optimization problem using the Gumbel-Softmax technique, QuantumDARTS enables joint gradient-based optimization of both circuit structure and continuous gate parameters. This approach avoids costly black-box search and allows efficient exploration of large circuit design spaces.

In this report, we reproduce and extend QuantumDARTS for the Max-Cut problem. We implement both the macro-search and micro-search variants following the original gate set, optimization strategy, and training procedure. Building on this baseline, we introduce a compiler-aware extension that incorporates practical hardware considerations into the architecture search. Specifically, we augment the training objective with penalties derived from circuit compilation metrics, including gate count, circuit depth, SWAP overhead, and estimated error rates obtained from an IBM quantum backend. Through experiments on small graph instances, we study how compiler-aware objectives influence the learned circuit structures and their performance under realistic noise conditions.

1.1 Approaches to Quantum Architecture Search

Existing approaches to quantum architecture search can be broadly categorized based on how circuit structures are explored and optimized.

Evolutionary and Genetic Algorithms represent early approaches to QAS, where quantum circuits are encoded as genomes and evolved using mutation and crossover operations. While these methods can discover simple circuits, their performance is sensitive to genome length and gate-set size, and they struggle to handle parameterized rotation gates efficiently.

Reinforcement Learning (RL) approaches model circuit construction as a sequential decision-making process in which an agent selects gates to maximize a reward signal. Although RL-based methods have demonstrated success on small-scale problems, they typically require extensive sampling and are computationally expensive.

Sampling-Based Methods search over a predefined set of candidate circuits by repeatedly sampling architectures and optimizing their parameters. These approaches often rely on non-differentiable sampling and predefined circuit templates, which can limit generality and scalability.

Differentiable Architecture Search methods relax discrete architecture choices into a continuous space, enabling end-to-end gradient-based optimization. QuantumDARTS follows this paradigm by employing the Gumbel-Softmax relaxation to enable differentiable gate selection, allowing efficient joint optimization of circuit structure and parameters without requiring predefined circuit templates.

2 Background

2.1 Max-Cut and Hamiltonian Formulation

The Max-Cut problem is defined on an undirected graph $G = (V, E)$, where the goal is to partition the vertex set V into two disjoint subsets such that the number of edges crossing the partition is maximized. This problem can be naturally mapped to a quantum optimization objective by associating a binary variable with each vertex and encoding the objective as a Hamiltonian.

The Max-Cut Hamiltonian used in this work is

$$H = \frac{1}{2} \sum_{(u,v)\in E} (I - Z_u Z_v)\,, \tag{1}$$

where Z_u and Z_v are Pauli-Z operators acting on qubits corresponding to vertices u and v. Measuring this Hamiltonian assigns a higher energy to bitstrings that place connected vertices in different partitions, making energy maximization equivalent to maximizing the cut value.

The initial quantum state is chosen as

$$|\psi_h\rangle = |+\rangle^{\otimes n}\,,$$

which corresponds to a uniform superposition over all possible vertex partitions. Circuit performance is evaluated using both the expectation value $\langle H \rangle$ and the Conditional Value-at-Risk (CVaR), computed over the top 20% of measurement outcomes ranked by cut value. CVaR emphasizes high-quality solutions rather than average performance.

2.2 QuantumDARTS Overview

QuantumDARTS is a differentiable quantum architecture search framework that automatically learns quantum circuit structures along with their continuous gate parameters. Instead of fixing a circuit template, QuantumDARTS defines a *search space* over possible gates at each circuit location and optimizes this space using gradient-based methods.

The candidate gate set used in this work follows the original paper and consists of

$$\mathcal{G} = \{\text{RzRyRz}, I, \text{CNOT}\}.$$

The single-qubit RzRyRz gate is parameterized by three rotation angles $(\theta_0, \theta_1, \theta_2)$, while CNOT provides entanglement between qubits.

Each circuit location (i, j) corresponding to qubit i at layer j, selects one gate from $\mathcal{G}$. This selection is represented by a discrete random variable M_{ij}, whose probability distribution is controlled by architecture parameters α_{ij}.

3 Methods

3.1 Differentiable Gate Selection

For an n-qubit circuit with m layers, the overall unitary is constructed as

$$\hat{U} = \prod_{j=1}^{m} \prod_{i=1}^{n} \hat{U}_{ij}, \tag{2}$$

where $\hat{U}_{ij} = \sigma(M_{ij})$ represents the unitary corresponding to the selected gate at position (i, j). The mapping $\sigma(\cdot)$ embeds a local gate into a $2^n \times 2^n$ unitary by applying identity operators on all other qubits.

The gate selection probabilities are defined as

$$\mathcal{T}_{ij}^{(k)} = \Pr\Big(M_{ij} = \mathcal{G}^{(k)}\Big) = \frac{\exp\Big(\alpha_{ij}^{(k)}\Big)}{\sum_{k'=1}^{K} \exp\Big(\alpha_{ij}^{(k')}\Big)}, \tag{3}$$

where $K = |\mathcal{G}|$. Since sampling from this discrete distribution is non-differentiable, QuantumDARTS uses the Gumbel-Softmax relaxation to enable gradient-based optimization.

In the forward pass, a single gate is sampled using a Gumbel perturbation, while in the backward pass, a softmax approximation is used:

$$\tilde{h}_{ij}^{(k)} = \frac{\exp\Big((\log \mathcal{T}_{ij}^{(k)} + G_k)/\tau\Big)}{\sum_{k'=1}^{K} \exp\Big((\log \mathcal{T}_{ij}^{(k')} + G_{k'})/\tau\Big)}, \tag{4}$$

where τ is a temperature parameter that is gradually annealed during training. As $\tau \to 0$, the distribution becomes increasingly discrete.

To reduce the number of architecture parameters, the logits α_{ij} are factorized as

$$\alpha_{ij} = P_{ij} Q_{ij},$$

where $P_{ij} \in \mathbb{R}^{1\times K'}$ and $Q_{ij} \in \mathbb{R}^{K'\times K}$.

3.2 Macro and Micro Architecture Search

In **macro-search**, a full n-qubit circuit is optimized, with independent gate choices at every qubit and layer. While expressive, this approach scales poorly with system size.

In **micro-search**, a shared two-qubit subcircuit with L_{sub} layers is learned and then tiled across all graph edges. This significantly reduces the search space and improves scalability, making micro-search suitable for larger graphs.

3.3 Optimization Procedure

Training alternates between optimizing continuous gate parameters θ and architecture parameters α. For fixed architecture parameters, θ is updated to minimize the loss

$$\mathcal{L}_\theta = -\langle \psi_h | U^\dagger H U | \psi_h \rangle. \tag{5}$$

For fixed gate parameters, α is updated using gradients through the Gumbel-Softmax relaxation. Temperature annealing is applied throughout training to encourage discrete gate selection.

Algorithm 1 summarizes the macro-search optimization procedure. After architecture search converges, the final discrete circuit is fixed and gate parameters are further optimized.

Algorithm 1 Macro Quantum Architecture Search (QuantumDARTS)

Require: Randomly initialized architecture parameters $\mathcal{P}, \mathcal{Q}$ and gate parameters θ, number of epochs N_{epoch}, inner-loop iterations N_{iter}

```
Initialize circuit unitary Û ← I_{2^n}
for epoch = 1 to N_epoch do
    Construct circuit unitary
    for layer = 1 to m do
        for qubit = 1 to n do
            Sample gate at position (qubit, layer) using Gumbel-Softmax (Eq. 3)
            Embed selected gate into Û_{qubit,layer}
            Update circuit: Û ← Û_{qubit,layer} · Û
        end for
    end for
    Optimize gate parameters
    for iter = 1 to N_iter do
        Compute loss L_θ = −⟨ψ_h|Û†HÛ|ψ_h⟩
        Update θ via gradient descent
        Recompute Û using updated θ
    end for
    Optimize architecture parameters
    Compute architecture loss L_A
    Update P, Q via gradient descent
end for
Fix final discrete architecture and optimize θ
```

3.4 Architecture Search Algorithm

QuantumDARTS performs architecture search by jointly optimizing circuit structure and gate parameters through an alternating optimization procedure. Algorithm 1 summarizes the macro-search variant used as the baseline in this work.

Algorithm 1 illustrates how the differentiable gate selection and optimization procedure is implemented in practice. At each epoch, a full circuit unitary is constructed by sampling gates at every qubit and layer using the Gumbel-Softmax relaxation. The optimization alternates between updating the continuous gate parameters θ, which control rotation angles, and the architecture parameters $(\mathcal{P}, \mathcal{Q})$, which determine the discrete gate choices. Temperature annealing is applied implicitly through the Gumbel-Softmax sampling process to gradually enforce discrete architectures.

4 Compiler-Aware Extension

This section describes our compiler-aware modification to the QuantumDARTS micro-search pipeline. The key idea is to preserve the original QuantumDARTS objective of maximizing Max-Cut performance, while additionally biasing the architecture search toward circuits that are more suitable for execution on real quantum hardware. This is achieved by periodically compiling the currently discovered circuit to a target IBM backend and extracting practical cost signals such as gate count, circuit depth, SWAP overhead, and estimated error rates. These signals are converted into penalties and incorporated directly into the architecture optimization objective.

4.1 Overview and Motivation

In standard QuantumDARTS, architecture search is guided solely by a quantum objective derived from the Max-Cut Hamiltonian, typically evaluated under ideal simulation. However, circuits that perform well in this idealized setting may become impractical after hardware compilation due to increased depth, additional routing operations, and accumulated gate errors. This discrepancy motivates the introduction of compiler feedback into the training loop.

Figure 1 illustrates the overall compiler-aware micro-search pipeline. A shared two-qubit subcircuit is learned using differentiable sampling over a candidate gate set. This subcircuit is then tiled across all edges of the input graph to form the full circuit. Periodically, the full circuit is compiled to a target IBM backend, and the resulting hardware-dependent metrics are injected into the loss function as penalties.

4.2 Multi-Objective Compiler-Aware Loss

Let H_c denote the Max-Cut Hamiltonian for a given graph instance and let $|\psi_h\rangle = |+\rangle^{\otimes n}$ be the initial state. For a candidate circuit unitary $\hat{U}_c$, the baseline QuantumDARTS objective minimizes the negative expected energy

$$\mathcal{L}_{\text{base}} = -\langle \psi_h | \hat{U}_c^\dagger H_c \hat{U}_c | \psi_h \rangle,$$

so that minimizing $\mathcal{L}_{\text{base}}$ corresponds to maximizing the expected cut value.

To incorporate compilation feedback, we define a compiler-aware loss

$$\mathcal{L}_{\text{comp}} = -\langle H_c \rangle + \lambda_1 \cdot P_{\text{gate}} + \lambda_2 \cdot P_{\text{swap}} + \lambda_3 \cdot P_{\text{error}}, \tag{6}$$

where each penalty term is computed from the *compiled* circuit:

- **Gate penalty P_{gate}:** the total number of gates in the compiled circuit, normalized by the baseline (compiler-unaware) gate count for the same problem size. Values below one indicate a reduction in gate count.

- **SWAP penalty P_{swap}:** the routing overhead, defined as

$$P_{\text{swap}} = \frac{\#\text{SWAPs}}{\#\text{total gates}},$$

  where #SWAPs includes both explicit SWAP gates and SWAPs introduced implicitly through routing and decomposition.

- **Error penalty P_{error}:** an estimated error proxy derived from backend calibration data, such as two-qubit gate error rates and readout errors, aggregated over the compiled circuit.

The coefficients $\lambda_1, \lambda_2, \lambda_3$ control the trade-off between solution quality and hardware efficiency. Setting all λ values to zero recovers the original compiler-unaware micro-search objective.

4.3 Compilation and Target Topology

All compiler-aware experiments target an IBM backend (IBM Torino) by enforcing its coupling map and basis gate set during transpilation. During training, the currently constructed circuit is periodically transpiled under these hardware constraints and the following quantities are extracted: (i) compiled gate counts, (ii) compiled circuit depth, (iii) routing and SWAP overhead, and (iv) estimated error signals from backend calibration properties.

To limit computational overhead, compilation is performed intermittently—every three epochs in our implementation. In epochs where compilation is not performed, the compiler penalty term is set to zero, and the algorithm reduces to standard QuantumDARTS micro-search.

4.4 Noise Model and Noisy Evaluation

To assess the robustness of discovered architectures under realistic noise, we construct a noise model from backend properties, including gate error rates and readout errors, and perform noisy simulation using Qiskit Aer. From the resulting measurement counts, we compute: (i) noisy CVaR over cut values, and (ii) a simple fidelity proxy defined as the probability mass of the most likely measured bitstring. While this proxy does not represent full state fidelity, it provides an interpretable indicator of output concentration under noise.

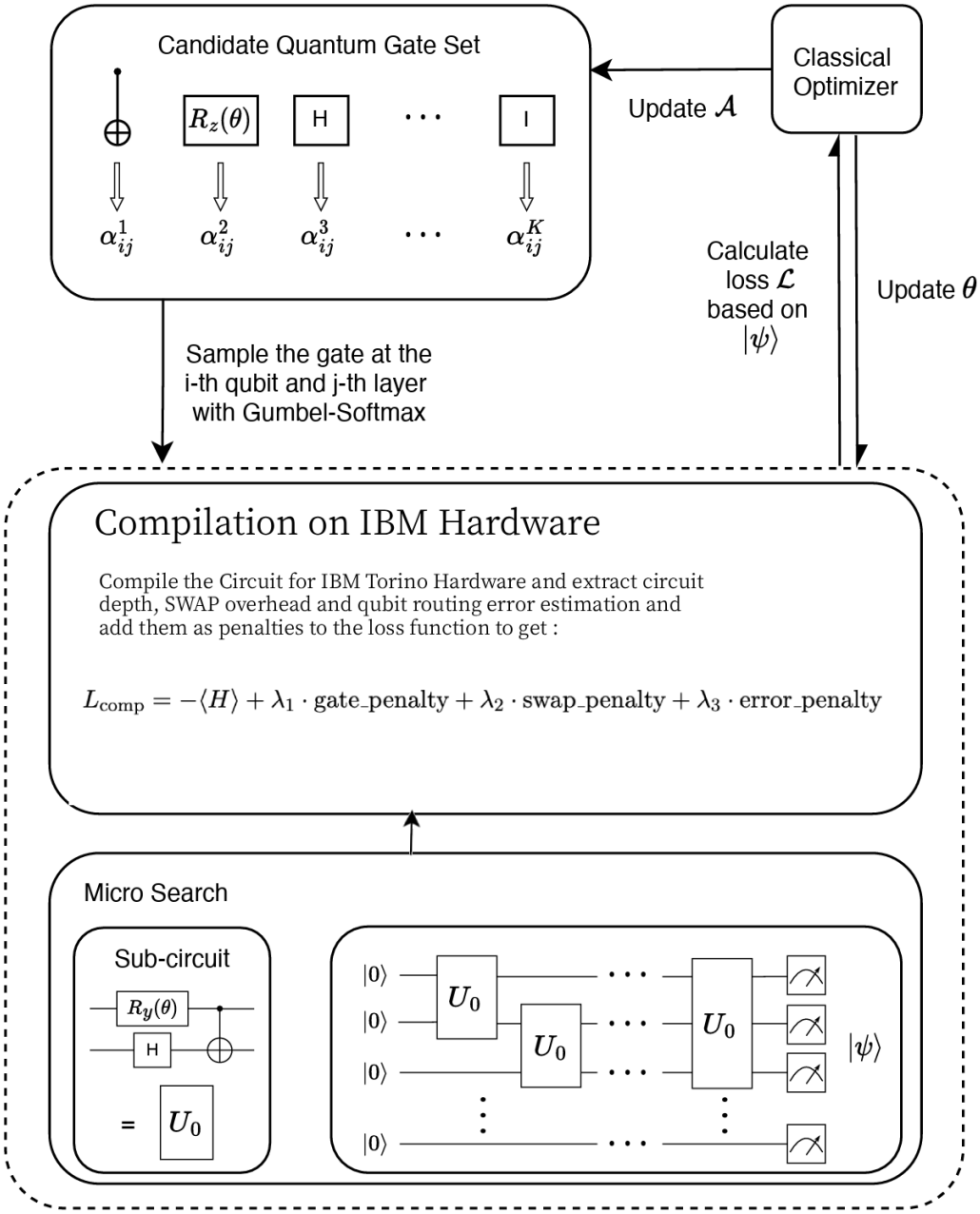

Figure 1: Compiler-aware micro-search architecture. A shared two-qubit subcircuit is learned via Gumbel-Softmax sampling and tiled across graph edges. The resulting circuit is periodically compiled to an IBM backend, and compilation-derived metrics are added as penalties to the training objective.

Algorithm 2 Micro Quantum Architecture Search with Compiler Awareness

Require: Graph $G = (V, E)$ with $n = |V|$ qubits; randomly initialized micro-architecture parameters $\mathcal{P}, \mathcal{Q}$; gate parameters θ; number of subcircuit layers L; epochs N_{epoch}; inner-loop iterations N_{iter}

Initialize shared two-qubit subcircuit unitary $\hat{U}_{\text{sub}} \leftarrow I_4$
for epoch $= 1$ to N_{epoch} **do**
    Construct shared two-qubit subcircuit
    $\hat{U}_{\text{sub}} \leftarrow I_4$
    for layer $= 1$ to L **do**
        for local qubit $q = 1$ to 2 **do**
            Sample gate at position (q, layer) using Gumbel-Softmax
            Embed selected gate into $\hat{U}_{q,\text{layer}}$
            Update subcircuit: $\hat{U}_{\text{sub}} \leftarrow \hat{U}_{q,\text{layer}} \cdot \hat{U}_{\text{sub}}$
        end for
    end for
    Tile subcircuit across graph edges
    Initialize global circuit $\hat{U}_c \leftarrow I_{2^n}$
    for each edge $(u, v) \in E$ **do**
        Embed $\hat{U}_{\text{sub}}$ on qubits (u, v) to form $\hat{U}_{uv}$
        Update circuit: $\hat{U}_c \leftarrow \hat{U}_{uv} \cdot \hat{U}_c$
    end for
    Optimize gate parameters
    for iter $= 1$ to N_{iter} **do**
        Compute loss $\mathcal{L}_\theta = -\langle \psi_h | \hat{U}_c^\dagger H_c \hat{U}_c | \psi_h \rangle$
        Update θ via gradient descent
        Recompute $\hat{U}_{\text{sub}}$ and $\hat{U}_c$
    end for
    Compiler-aware evaluation
    if *epoch* mod $3 = 0$ **then**
        Compile $\hat{U}_c$ to target IBM hardware
        Extract gate count, SWAP overhead, and error estimates
        $\mathcal{L}_{\text{comp}} \leftarrow \lambda_1 P_{\text{gate}} + \lambda_2 P_{\text{swap}} + \lambda_3 P_{\text{error}}$
    else
        $\mathcal{L}_{\text{comp}} \leftarrow 0$
    end if
    Optimize architecture parameters
    $\mathcal{L}_{\mathcal{A}} = -\langle \psi_h | \hat{U}_c^\dagger H_c \hat{U}_c | \psi_h \rangle + \mathcal{L}_{\text{comp}}$
    Update $\mathcal{P}, \mathcal{Q}$ via gradient descent
end for
Fix final shared subcircuit and perform final fine-tuning of θ

4.5 Compiler-Aware Micro-Search Procedure

Algorithm 2 summarizes the complete training loop for compiler-aware micro-search. The algorithm follows the micro-search variant of QuantumDARTS, in which a *single shared two-qubit subcircuit* is learned and reused across the entire problem graph, while extending it with periodic compiler feedback.

At the beginning of each epoch, the algorithm constructs the shared two-qubit subcircuit $\hat{U}_{\text{sub}}$ (Lines 1–9 of Algorithm 2). For each layer and each local qubit position within the subcircuit, a gate is selected from the candidate gate set using Gumbel-Softmax sampling parameterized by the architecture logits $\mathcal{P}$ and $\mathcal{Q}$. The sampled gates are sequentially multiplied to form the subcircuit unitary.

Once the shared subcircuit is constructed, it is *tiled across the graph* (Lines 10–14). For each edge $(u, v) \in E$, the two-qubit subcircuit is embedded into the full n-qubit Hilbert space on qubits (u, v), and all such embedded unitaries are multiplied to obtain the global circuit unitary $\hat{U}_c$. This tiling strategy allows the architecture search to scale with graph size while keeping the number of trainable architecture parameters fixed.

The algorithm then performs an inner optimization loop over the continuous gate parameters θ (Lines 15–19). In this phase, the architecture is held fixed and θ is updated to minimize the negative expectation value of the Max-Cut Hamiltonian,

$$\mathcal{L}_\theta = -\langle \psi_h | \hat{U}_c^\dagger H_c \hat{U}_c | \psi_h \rangle,$$

which corresponds to maximizing the expected cut value. After each update, the circuit is rebuilt using the updated parameters.

Every few epochs (every three epochs in our implementation), the algorithm invokes a *compiler-aware evaluation step* (Lines 20–26). The current circuit $\hat{U}_c$ is compiled to the target IBM backend, enforcing the backend's coupling map and basis gates. From the compiled circuit, hardware-relevant metrics—including gate count, SWAP overhead, and estimated error—are extracted and converted into penalty terms. These penalties are accumulated into a compiler-aware loss component $\mathcal{L}_{\text{comp}}$ as defined in Eq. (6). In epochs where compilation is skipped, this penalty term is set to zero, reducing the algorithm to standard micro-search.

Finally, the architecture parameters $\mathcal{P}$ and $\mathcal{Q}$ are updated (Lines 27–32) using the combined objective

$$\mathcal{L}_A = -\langle \psi_h | \hat{U}_c^\dagger H_c \hat{U}_c | \psi_h \rangle + \mathcal{L}_{\text{comp}},$$

which balances Max-Cut performance with hardware efficiency. After all training epochs are complete, the final discrete subcircuit architecture is fixed and the gate parameters θ are fine-tuned.

Overall, Algorithm 2 integrates compiler feedback directly into the architecture optimization loop while preserving the original QuantumDARTS micro-search structure. This allows the search process to explicitly trade off solution quality against realistic hardware costs.

4.6 Explanation of the Architecture Diagram

Figure 1 depicts the end-to-end data flow of the compiler-aware micro-search framework. The upper portion of the diagram shows the candidate gate set and architecture logits, which define a probability distribution over discrete gate choices. Gumbel-Softmax sampling selects a single gate in the forward pass while allowing gradients to propagate during backpropagation.

The central addition is the compilation stage, where the currently instantiated circuit is transpiled for the target IBM backend. This step enforces hardware connectivity and basis constraints and exposes practical costs such as circuit depth and SWAP insertion. These costs are transformed into penalty terms and added to the loss function in Eq. (6).

The lower portion of the diagram illustrates micro-search: a single learned two-qubit subcircuit is reused across all edges of the graph, enabling scalable architecture discovery. Importantly, compiler-aware penalties are applied *after tiling*, ensuring that the optimization reflects the cost of the full circuit that would be executed on hardware.

5 Implementation Highlights

This section summarizes key implementation details and design choices that were critical for faithfully reproducing QuantumDARTS and for enabling the proposed compiler-aware extension.

Gate Set. We strictly follow the gate set used in the original QuantumDARTS paper. The candidate gate library consists of the single-qubit rotation block Rz–Ry–Rz parameterized by three continuous angles, the identity gate I, and the two-qubit CNOT gate. Restricting the gate set in this way ensures comparability with prior results and avoids introducing additional expressivity not present in the original method.

Differentiable Gate Sampling. Discrete gate selection is made differentiable using the Gumbel-Softmax trick. In the forward pass, a hard (one-hot) gate choice is sampled to preserve physical circuit semantics, while in the backward pass a soft relaxation is used to allow gradients to flow through the architecture parameters. This enables end-to-end gradient-based optimization over circuit structure.

Architecture Parameterization. Architecture logits α_{ij} are factorized using the low-rank decomposition $\alpha_{ij} = P_{ij}Q_{ij}$. This decomposition significantly reduces the number of trainable parameters while stabilizing optimization, as originally proposed in QuantumDARTS. We maintain separate parameters for architecture selection and gate rotation angles.

Temperature Annealing. The Gumbel-Softmax temperature τ is annealed according to a fixed schedule, gradually decreasing toward a minimum value $\tau_{\min}$. Early in training, higher temperatures encourage exploration over the gate set, while lower temperatures later in training promote near-discrete architecture decisions.

CVaR-Based Evaluation. In addition to expectation values, circuit quality is evaluated using Conditional Value-at-Risk (CVaR) over the top 20% of measured cut values. CVaR emphasizes high-quality solutions and provides a more robust performance metric than raw expectation values, particularly for variational algorithms with broad output distributions. Care is taken to ensure numerical stability when computing CVaR from finite-sample measurement counts.

Qiskit Integration. All circuits are converted to Qiskit representations for visualization, transpilation, and noisy simulation. This allows direct access to IBM backend properties, including coupling maps, basis gates, and calibration data, and ensures that compiler-aware metrics are extracted under realistic hardware constraints.

Compiler-Aware Bookkeeping. During compiler-aware micro-search, gate count, circuit depth, SWAP overhead, and error proxies are recorded at each compilation interval. These metrics are logged alongside training losses and CVaR values, enabling systematic comparison between compiler-unaware and compiler-aware architectures and facilitating post-training analysis.

6 Experiments and Results

We begin with a sanity-check evaluation of QuantumDARTS on a small Max-Cut instance, prior to introducing compiler-aware penalties. The goal of this experiment is to verify that both the *macro-*

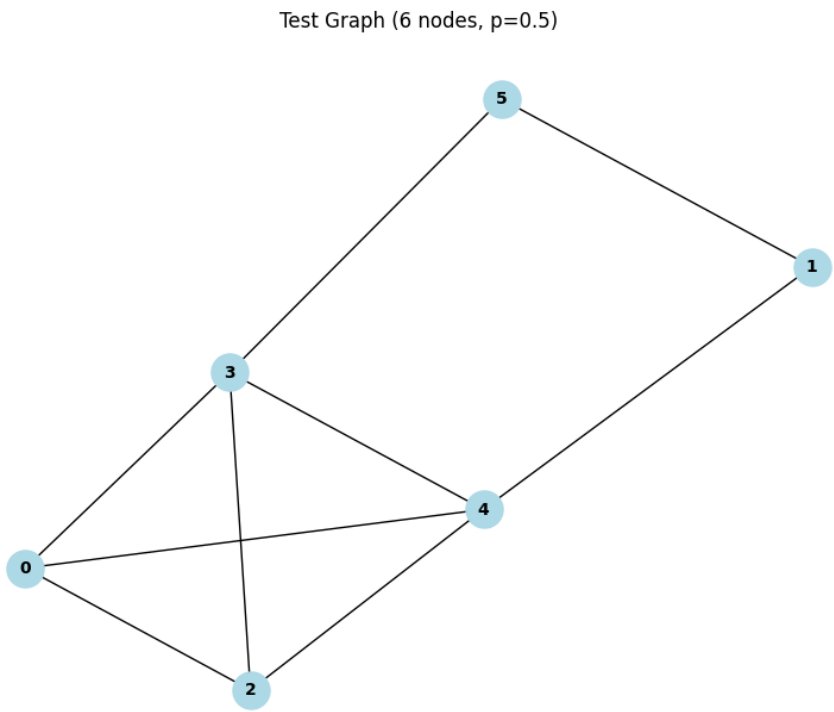

Figure 2: Test graph used for the macro-search sanity check: 6-node Erdős–Rényi instance with $p = 0.5$.

search and *micro-search* variants of QuantumDARTS are capable of recovering optimal solutions on small graphs under ideal (noise-free, compiler-unaware) conditions.

In this subsection, we focus on results obtained using **macro quantum architecture search**. Results for micro-search are presented in the subsequent subsection.

6.1 Macro Search: Sanity Check on a 6-Node Graph

Experimental Setup. We consider a 6-node Erdős–Rényi graph with edge probability $p = 0.5$. The corresponding Max-Cut Hamiltonian is constructed as

$$H_c = \frac{1}{2} \sum_{(u,v)\in E} (I - Z_u Z_v)\,,$$

and the initial state is fixed to $|\psi_h\rangle = |+\rangle^{\otimes 6}$. Architecture search is performed using the original QuantumDARTS macro-search algorithm, without any compiler-aware penalties or noise modeling.

Test Graph and Optimal Solution. Figure 2 shows the generated 6-node test graph. The optimal classical Max-Cut solution has cut value 7, as illustrated in Figure 3. This optimal value serves as a ground-truth reference for evaluating the quantum solution.

Quantum Solution Quality. After training, the macro-search architecture successfully recovers a quantum solution achieving the *optimal cut value of* 7. Figure 4 visualizes the partition corre-

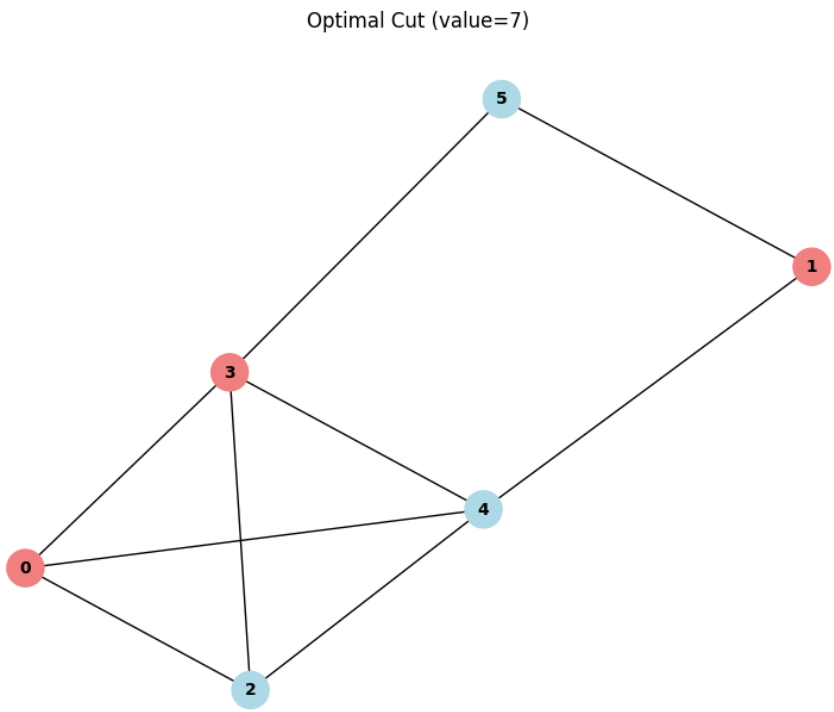

Figure 3: Classical optimal Max-Cut partition for the test graph (cut value = 7).

sponding to the most probable measured bitstring from the trained circuit, which matches the optimal classical cut.

The extracted bitstring and statistics are:

- Most probable bitstring: $[1, 1, 0, 1, 0, 0]$
- Cut value from quantum solution: 7
- Solution probability: 0.0418

Training Dynamics. Figure 5 shows the training evolution of the optimization process. The expectation value $\langle H_c \rangle$ steadily increases during training, while the loss decreases, indicating successful variational optimization. Figure 5 reports the CVaR (top 20%) objective, which remains close to the optimal cut value throughout training, confirming that high-quality cuts are consistently sampled.

The Gumbel-Softmax temperature schedule used during architecture search is shown in Figure 5, illustrating the gradual transition from soft exploration to near-discrete gate selection.

Learned Circuit Structure. Figure 6 presents the final learned macro-search circuit. The circuit consists of 60 total gates with a depth of 10, and uses only the allowed gate set $\{\text{RzRyRz}, I, \text{CNOT}\}$, consistent with the original QuantumDARTS formulation.

Summary of Macro Search Results. Table 1 summarizes the key quantitative results obtained from macro-search on the 6-node graph.

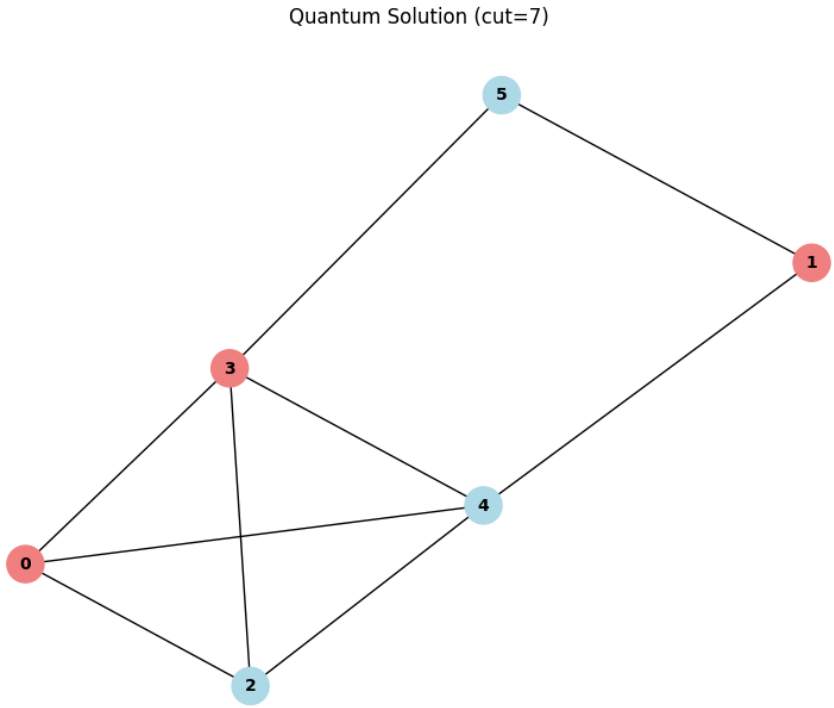

Figure 4: Macro-search quantum solution visualized using the most probable measured bitstring (cut value = 7).

Discussion. These results confirm that the macro-search variant of QuantumDARTS is able to discover architectures that achieve optimal Max-Cut solutions on small graph instances under idealized conditions. This validates the correctness of our implementation and provides a strong baseline for comparison with micro-search and compiler-aware extensions, which we examine next.

6.2 Micro-Search Sanity Check (6-node Erdős–Rényi, p=0.5)

We next ran a small-scale sanity check for the *micro-search* variant on a 6-node Erdős–Rényi graph with edge probability $p = 0.5$ (same setting as the macro-search check). In micro-search, the architecture search does not learn an independent gate choice at every (qubit, layer) location of the full n-qubit circuit. Instead, it learns a *shared two-qubit subcircuit* (with L sub-layers) and then *tiles* this subcircuit across the graph edges to form the full circuit. Concretely, if $\hat{U}_{\text{sub}}$ is the learned two-qubit unitary, then the global circuit unitary is constructed as

$$\hat{U}_c = \prod_{(u,v)\in E} \hat{U}_{uv},$$

where each $\hat{U}_{uv}$ denotes the embedding of the same $\hat{U}_{\text{sub}}$ onto the pair of qubits (u, v) (and identity on all other qubits). Thus, the same learned subcircuit block is *reused* once per edge, yielding a circuit whose structure explicitly repeats across edges.

Figure 7 summarizes the training dynamics for micro-search (losses, Max-Cut expectation, CVaR, and temperature schedule), and Figure 8 visualizes the learned circuit instance produced by tiling the shared subcircuit over the input graph.

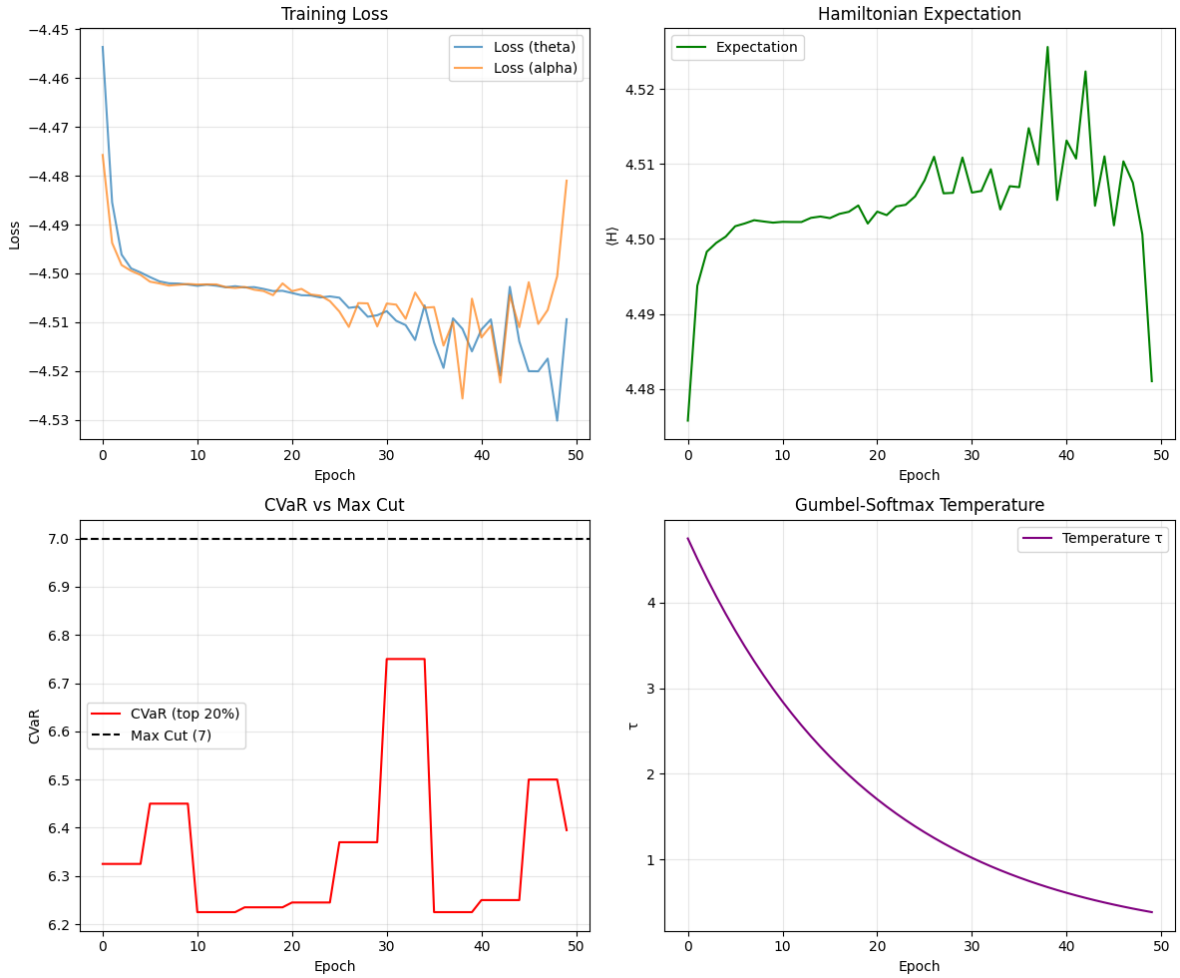

Figure 5: Macro-search training curves on the 6-node graph: loss values, Hamiltonian expectation $\langle H_c \rangle$, CVaR (top 20%), and Gumbel-Softmax temperature annealing.

Table 2 reports the key micro-search outcome for this sanity check. The micro-search solution matches the classical optimum Max-Cut value of 7 for this graph instance. In addition to the best cut extracted from the most-probable output bitstring, we report CVaR at the top 20% of sampled cuts as an aggregate performance indicator.

Discussion. This run serves as a functional correctness check for the micro-search implementation: (i) the training procedure produces a valid circuit, (ii) the CVaR metric is consistent with near-optimal cuts, and (iii) the extracted most-likely bitstring achieves the optimal Max-Cut value for the instance. Importantly, the circuit construction in Figure 8 reflects the defining micro-search property—a repeated shared two-qubit subcircuit that is embedded and applied once per graph edge.

6.3 Compiler-Aware Micro-Search on 6-Node Erdős–Rényi Graphs

We evaluated the compiler-aware micro-search loss (Sec. 4) on three 6-node Erdős–Rényi (ER) instances with edge probabilities $p \in \{0.25, 0.5, 0.75\}$. For each instance, we compare a baseline

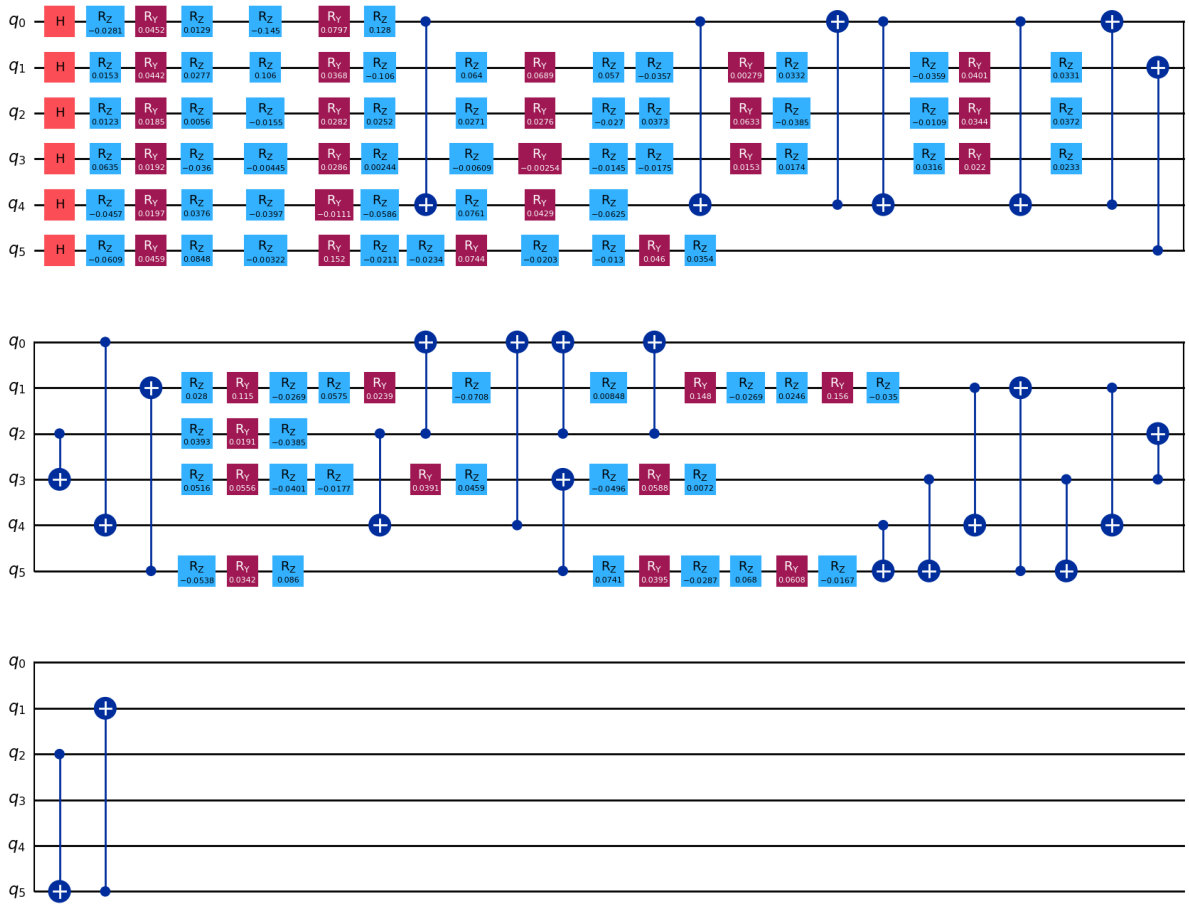

Figure 6: Final macro-search circuit learned for the 6-node test graph.

micro-search run (no compiler penalties) against the compiler-aware variant. Metrics are computed from the transpiled circuit (gate count, depth, SWAP count) and from evaluation runs (CVaR and a noisy "fidelity" proxy defined as the probability mass of the most likely output bitstring under the Aer noise model). Across all three graphs, SWAP count remained 0 after transpilation, while gate count and depth typically increased under compiler awareness. CVaR and noisy metrics changed only marginally, indicating limited benefit from the current penalty design/weights at this problem size.

6.3.1 p=0.5 (ER-6)

For $p = 0.5$, compiler awareness slightly increased compiled gate count and depth, while CVaR remained essentially unchanged (Fig. 9). The noisy fidelity proxy decreased, consistent with the modest increase in expected error rate (Tab. 3).

6.3.2 p=0.25 (ER-6)

For $p = 0.25$, compiler awareness produced a substantially larger compiled circuit (higher gate count and depth) without improving CVaR (Fig. 10, Tab. 4). The expected error proxy increased noticeably, and the noisy fidelity proxy decreased slightly.

Table 1: Macro-search results on a 6-node Erdős–Rényi graph ($p = 0.5$).

Metric	**Value**
Number of nodes	6
Classical Max-Cut	7
Quantum cut value	7
CVaR (top 20%)	6.54
Circuit depth	10
Total gates	60
Optimal solution found	Yes

Table 2: Micro-search sanity check results on a 6-node ER graph ($p = 0.5$).

Metric	Value
Classical Max-Cut (optimal)	7
CVaR (top 20%)	6.5160
Best bitstring (most probable)	$[0, 1, 1, 1, 0, 0]$
Partition ($x \in \{-1, +1\}^n$)	$[-1, 1, 1, 1, -1, -1]$
Cut value from extracted solution	7
Solution probability	0.041106
Found optimal solution	True

6.3.3 p=0.75 (ER-6)

For $p = 0.75$, CVaR (ideal and noisy) remained effectively unchanged, while compiler awareness again increased the compiled gate count and slightly increased depth (Fig. 11, Tab. 5). No SWAPs were introduced after transpilation for either variant.

Takeaway. On these 6-node ER instances, the compiler-aware penalty terms did not yield clear improvements in cut quality (CVaR) or noise robustness. In several cases, the compiled circuit became larger (more gates/depth) and the expected error proxy increased, which can offset any minor changes in the learned architecture.

6.4 Compiler-Aware Evaluation on 8-Node ER Graphs

We next evaluate the effect of compiler awareness on larger Erdős–Rényi (ER) instances with $n = 8$ nodes at edge densities $p \in \{0.25, 0.5, 0.75\}$. Compared to the 6-node sanity checks, 8-node experiments are computationally more demanding: increasing the number of layers substantially increases the cost of repeated unitary construction/multiplication in our implementation. Therefore, for $n = 8$ we used fewer layers than would be ideal, and as a result the algorithm does not necessarily reach the true Max-Cut optimum for every instance.

Table 6 aggregates the final compiled metrics and performance indicators. For $p = 0.5$ we observe a clear compiler-side improvement: the compiler-aware variant reduces compiled gate count ($281 \to 217$) and depth ($127 \to 88$), while keeping noisy CVaR essentially unchanged (10.59 in both cases) and slightly improving the expected error proxy ($0.1566 \to 0.0924$). For $p = 0.25$, the gains are milder: depth reduces ($125 \to 97$) with nearly identical gate count ($281 \to 280$) and similar noisy CVaR (8.29 in both), while the expected error proxy improves slightly ($0.1346 \to 0.1319$). In contrast, for $p = 0.75$ the compiler-aware run collapses to a much less hardware-friendly circuit

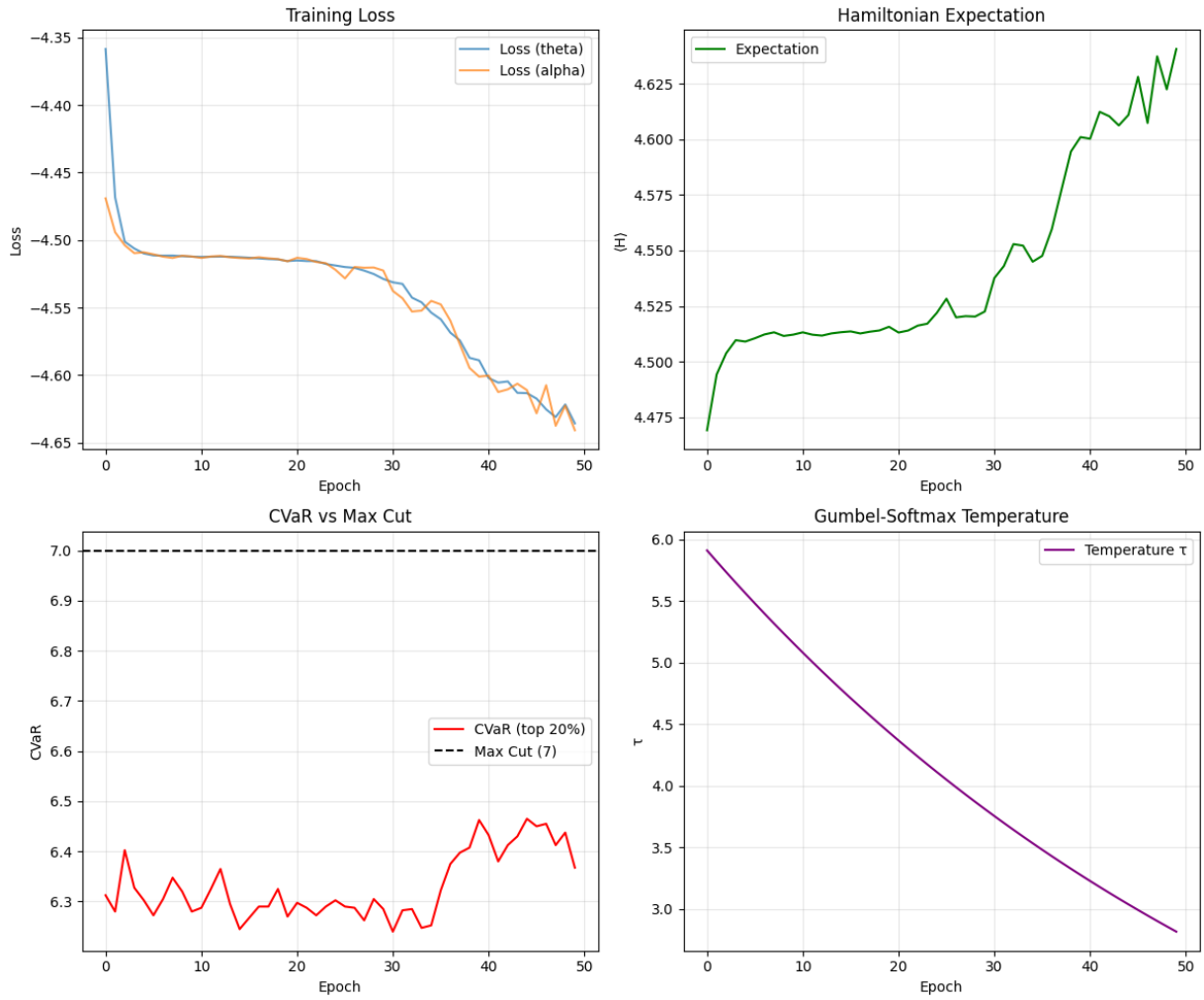

Figure 7: Micro-search training curves on a 6-node ER graph ($p = 0.5$). We plot the optimization losses for θ and architecture parameters, the Max-Cut Hamiltonian expectation, CVaR (top 20% cuts), and the Gumbel-Softmax temperature schedule..

(gate count $40 \to 378$ and depth $5 \to 138$) with a large increase in expected error proxy ($0.0039 \to 0.2062$), indicating that the penalty signals did not reliably steer optimization in this dense regime under the reduced-layer setting.

Overall, these 8-node results suggest that compiler awareness can meaningfully improve compiled resource metrics (especially depth and gate count) for some instances (notably $p = 0.5$ and partially $p = 0.25$), but the behavior is not uniform across densities. In the dense case ($p = 0.75$), the penalty signals did not prevent convergence to a substantially more expensive compiled circuit. A likely contributing factor is that the reduced-layer setting (required to avoid unitary blowup) limits the expressive capacity and may increase optimization instability, particularly in regimes where the number of tiled interactions grows quickly with density.

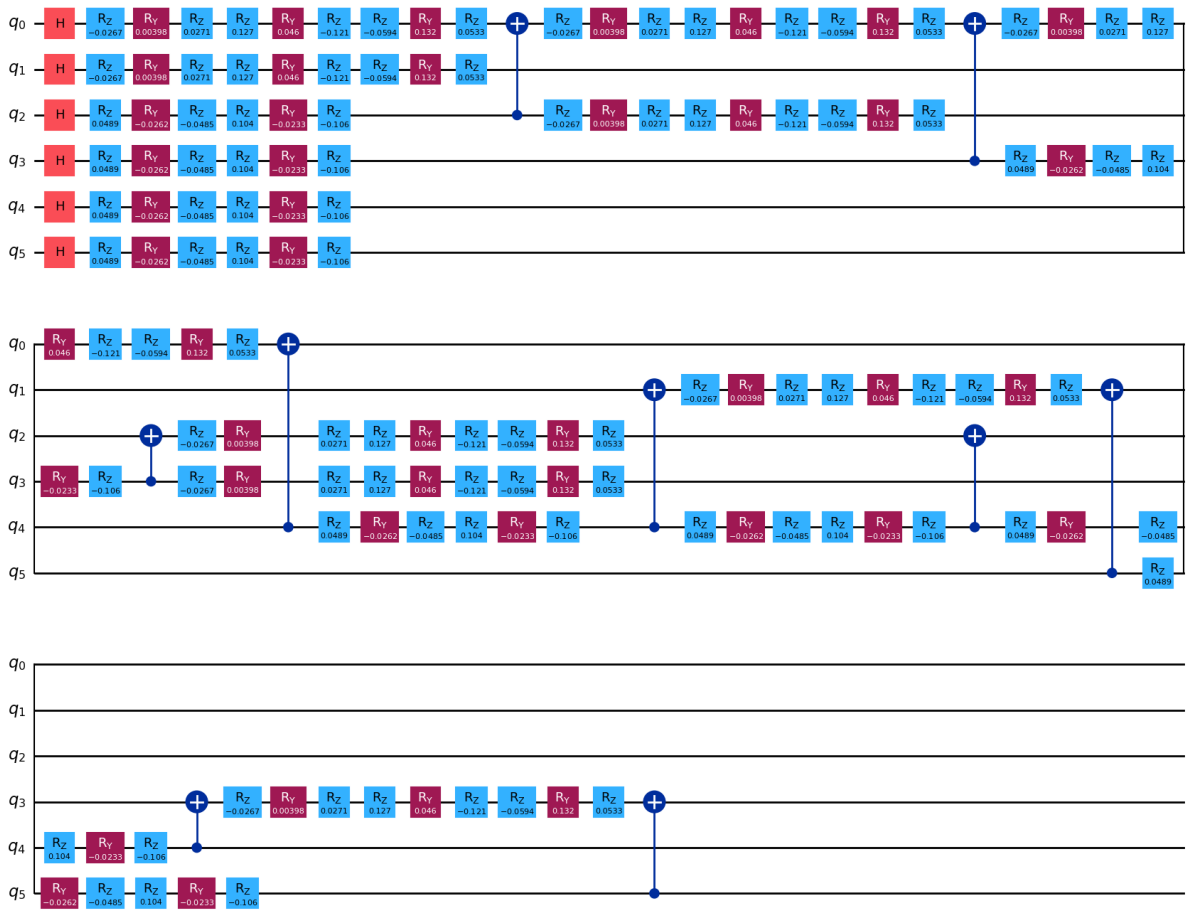

Figure 8: Learned micro-search circuit visualization for the 6-node ER instance ($p = 0.5$). The circuit is formed by repeating the same learned two-qubit subcircuit across all edges $(u, v) \in E$ via embedding.

7 Conclusion and Future Work

In this work, we studied *compiler-aware quantum architecture search* by extending the fully differentiable QuantumDARTS framework with hardware-driven penalties derived from circuit compilation. QuantumDARTS employs Gumbel–Softmax relaxation to enable gradient-based optimization over discrete gate choices, allowing both macro-level and micro-level quantum architectures to be learned end-to-end.

Our experimental evaluation leads to several key conclusions. First, macro-level architecture search is computationally expensive for Max-Cut instances and does not scale favorably even for modest graph sizes, making it impractical for repeated experimentation. In contrast, micro-search exhibits improved scalability by learning a compact, reusable two-qubit subcircuit that is tiled across graph edges, enabling effective optimization for Max-Cut while controlling parameter growth.

Second, incorporating compiler awareness into micro-search does not yield measurable benefits for very small problem instances (e.g., 6-node graphs), where baseline circuits are already shallow and well within hardware constraints. However, for larger instances (8-node graphs), compiler-aware micro-search demonstrates promising behavior: in two out of three tested edge-density regimes,

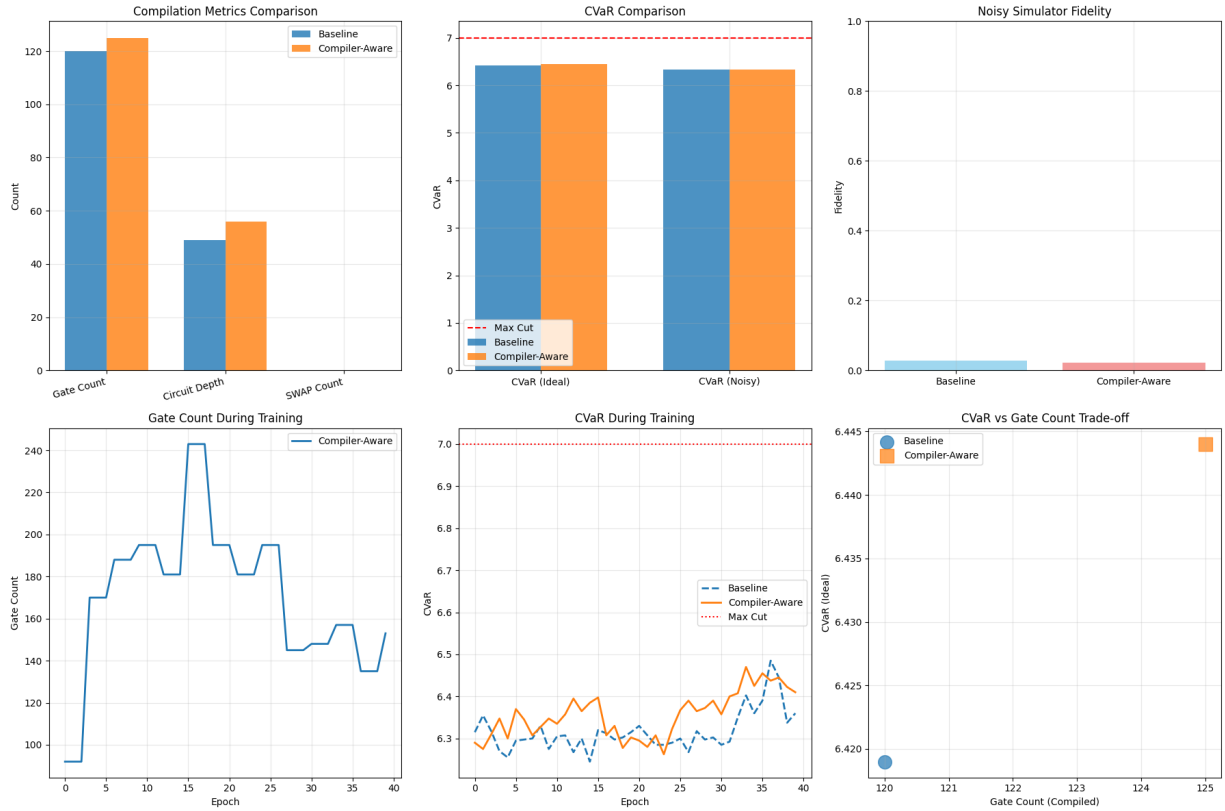

Figure 9: Baseline vs. compiler-aware micro-search on a 6-node ER graph with $p = 0.5$. The figure summarizes compiled circuit metrics, CVaR (ideal/noisy), noisy fidelity proxy, and training traces.

the compiler-aware objective produced circuits with significantly reduced compiled gate count and depth while maintaining comparable CVaR performance under both ideal and noisy simulation. These results suggest that compiler awareness becomes more relevant as circuit complexity and hardware pressure increase.

At the same time, our study highlights important limitations. For dense graphs and reduced-depth settings, compiler penalties alone are not always sufficient to steer optimization toward hardware-efficient solutions, occasionally leading to worse compiled circuits. Moreover, for 8-node graphs the true Max-Cut value was not always reached, as we intentionally limited the number of circuit layers. Increasing circuit depth would improve expressivity, but doing so leads to prohibitively expensive unitary construction and multiplication during statevector simulation on our system. As a result, we did not evaluate larger graphs ($n > 8$), since statevector-based simulation becomes infeasible due to exponential memory and computation growth.

Future Work

Several directions remain open for future investigation. A natural extension is to evaluate compiler-aware micro-search on larger graphs using more scalable simulation strategies, such as tensor-network simulators, sampling-based estimators, or direct execution on quantum hardware. Running experiments across multiple random graph instances for each (n, p) configuration would also allow for statistically robust conclusions.

Additionally, future work could explore alternative formulations of compiler-aware objectives,

Table 3: Compiler-awareness comparison on ER-6 ($p = 0.5$). Reported circuit metrics are after transpilation; CVaR is computed over cut values (top 20%).

Metric	**Baseline**	**Compiler-Aware**	**Δ (CA − Base)**
Gate Count	120	125	+5
Circuit Depth	49	56	+7
SWAP Count (optimized)	0	0	0
CVaR (Ideal)	6.42	6.44	+0.02
CVaR (Noisy)	6.33	6.33	0.00
Fidelity (Noisy)	0.0290	0.0218	−0.0072
Expected Error Rate	0.0512	0.0551	+0.0039

Table 4: Compiler-awareness comparison on ER-6 ($p = 0.25$).

Metric	**Baseline**	**Compiler-Aware**	**Δ (CA − Base)**
Gate Count	94	131	+37
Circuit Depth	43	69	+26
SWAP Count (optimized)	0	0	0
CVaR (Ideal)	5.20	5.14	−0.06
CVaR (Noisy)	5.17	5.17	0.00
Fidelity (Noisy)	0.0278	0.0248	−0.0030
Expected Error Rate	0.0349	0.0553	+0.0204

including more refined error models, connectivity-aware penalties, or adaptive weighting schedules for compiler costs during training. Finally, the proposed compiler-aware framework is not limited to Max-Cut or QuantumDARTS; applying similar ideas to other parameterized quantum circuits, variational algorithms, or problem domains represents an important avenue for further research.

References

- **QuantumDARTS (core method).** Y. Du, T. Huang, S. You, M.-H. Hsieh, and D. Tao, *Quantum circuit architecture search for variational quantum algorithms*, npj Quantum Information, 8(1):1–8, 2022.

- **Differentiable architecture search with Gumbel-Softmax.** J. Chang, X. Zhang, Y. Guo, G. Meng, S. Xiang, and C. Pan, *Differentiable architecture search with ensemble Gumbel-Softmax*, CoRR, abs/1905.01786, 2019.

- **Gumbel-Softmax reparameterization.** E. Jang, S. Gu, and B. Poole, *Categorical reparameterization with Gumbel-Softmax*, International Conference on Learning Representations (ICLR), 2017.

- **Variational Quantum Algorithms (background).** A. Peruzzo et al., *A variational eigenvalue solver on a photonic quantum processor*, Nature Communications, 5:4213, 2014.

- **QAOA and Max-Cut benchmark.** E. Farhi, J. Goldstone, and S. Gutmann, *A quantum approximate optimization algorithm*, arXiv:1411.4028, 2014.

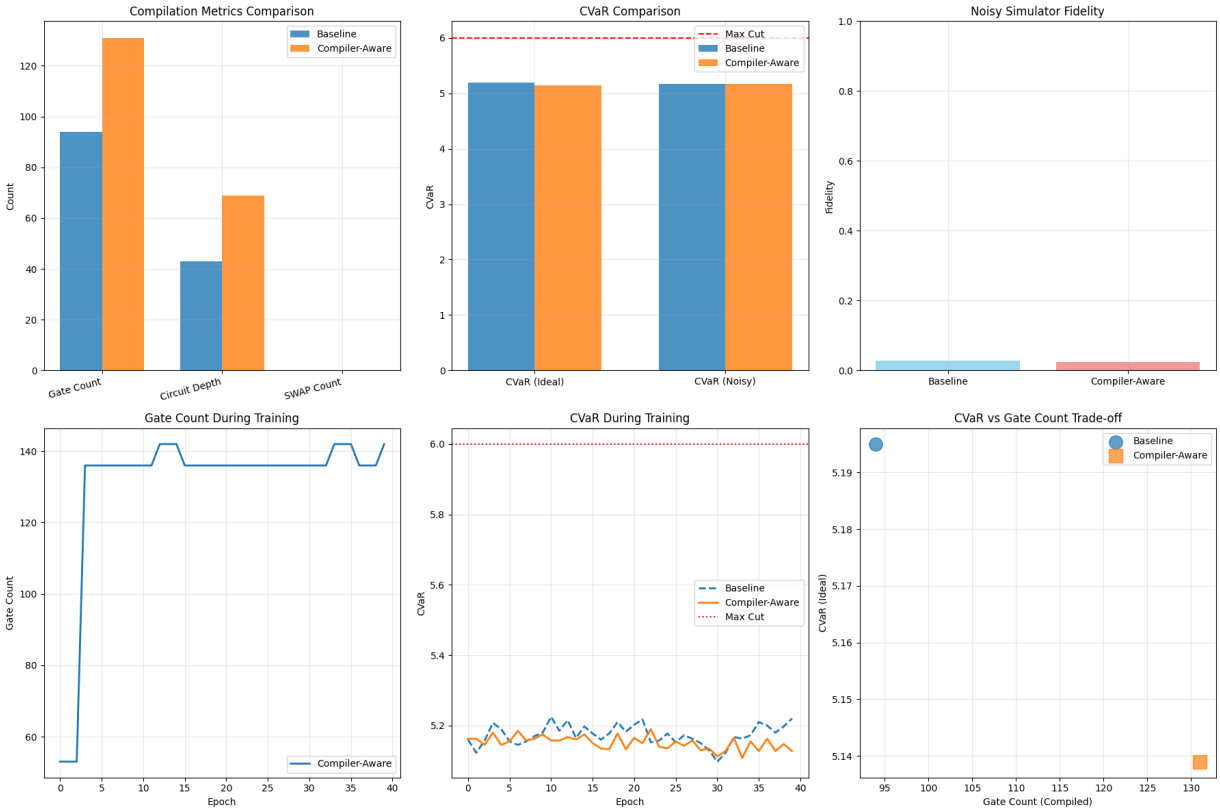

Figure 10: Baseline vs. compiler-aware micro-search on a 6-node ER graph with $p = 0.25$.

- **CVaR for variational quantum optimization.** P. K. Barkoutsos et al., *Improving variational quantum optimization using CVaR*, Quantum, 4:256, 2020.
- **Qiskit framework and transpilation.** G. Aleksandrowicz et al., *Qiskit: An open-source framework for quantum computing*, Accessed: Mar. 16, 2019.
- **Noise modeling and simulation.** J. R. McClean et al., *OpenFermion: The electronic structure package for quantum computers*, Quantum Science and Technology, 5(3):034014, 2020.
- **Random graph generation (Erdős–Rényi).** P. Erdős and A. Rényi, *On the evolution of random graphs*, Publications of the Mathematical Institute of the Hungarian Academy of Sciences, 5:17–60, 1960.
- **Quantum circuit compilation and hardware efficiency.** H. L. Tang et al., *qubit-adapt-vqe: An adaptive algorithm for constructing hardware-efficient ansätze*, PRX Quantum, 2(2):020310, 2021.

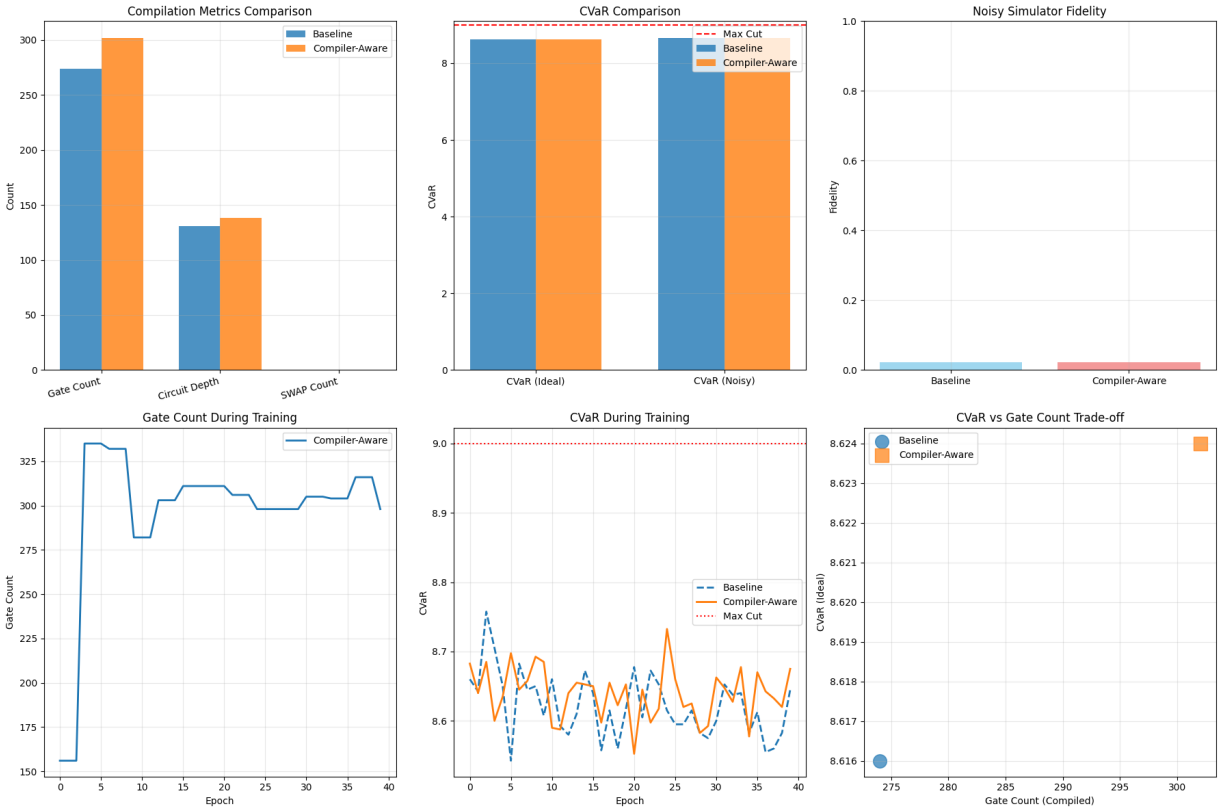

Figure 11: Baseline vs. compiler-aware micro-search on a 6-node ER graph with $p = 0.75$.

Table 5: Compiler-awareness comparison on ER-6 ($p = 0.75$).

Metric	**Baseline**	**Compiler-Aware**	**Δ (CA − Base)**
Gate Count	274	302	+28
Circuit Depth	131	138	+7
SWAP Count (optimized)	0	0	0
CVaR (Ideal)	8.62	8.62	+0.01
CVaR (Noisy)	8.67	8.67	0.00
Fidelity (Noisy)	0.0224	0.0214	−0.0010
Expected Error Rate	0.1421	0.1585	+0.0164

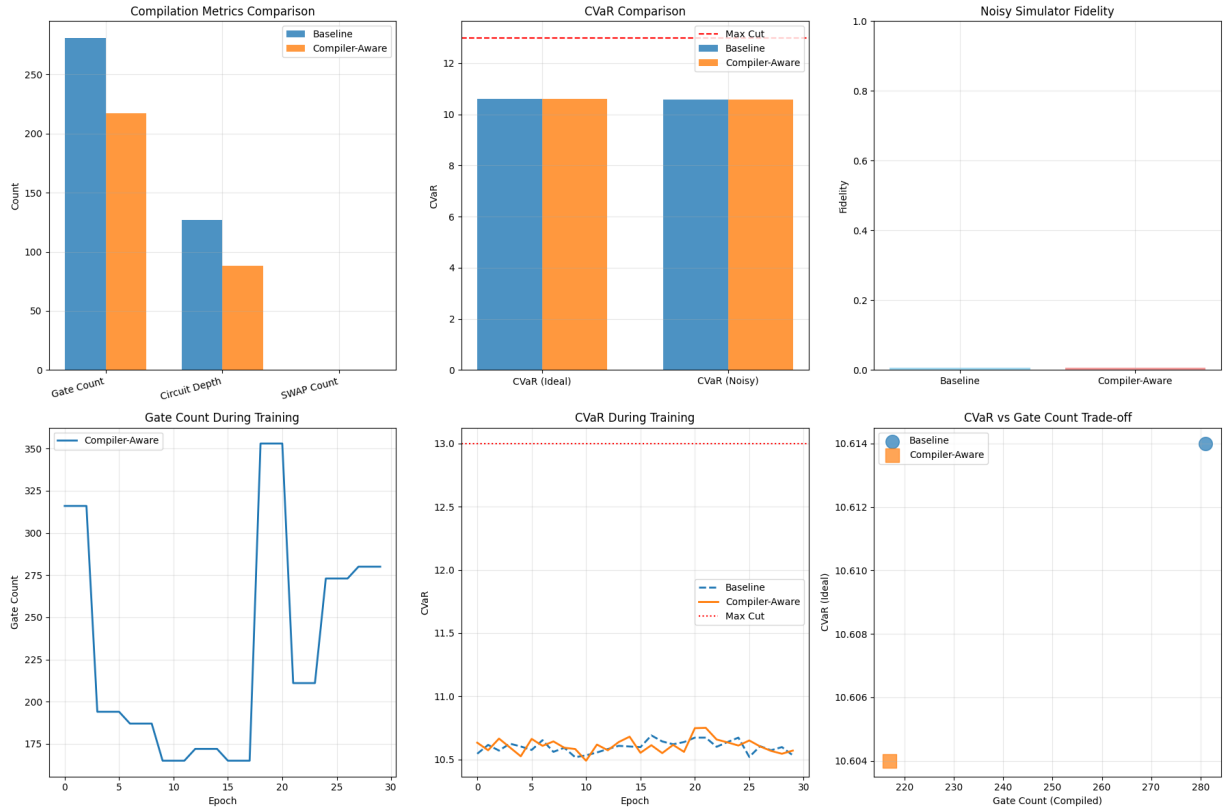

Figure 12: Compiler-aware vs. baseline comparison for an 8-node ER instance with $p = 0.5$. The plot summarizes compiled resource metrics (gate count/depth), CVaR (ideal/noisy), noisy fidelity proxy, and training trends.

Table 6: Summary of compiler-aware evaluation on 8-node ER graphs. "Fidelity (noisy)" is the probability mass of the most likely measured bitstring (a simple distribution-concentration proxy). "Expected error rate" is the backend-derived error proxy aggregated over the compiled circuit.

ER density	**Gate count**		**Depth**		**CVaR (Ideal)**		**CVaR (Noisy)**	
	Base	CA	Base	CA	Base	CA	Base	CA
$p = 0.50$	281	217	127	88	10.61	10.60	10.59	10.59
$p = 0.25$	281	280	125	97	8.38	8.31	8.29	8.29
$p = 0.75$	40	378	5	138	14.38	14.35	14.35	14.35
ER density	**SWAP count**		**Fidelity (Noisy)**		**Expected error rate**			
	Base	CA	Base	CA	Base	CA		
$p = 0.50$	0	0	0.0068	0.0066	0.1566	0.0924		
$p = 0.25$	0	0	0.0070	0.0070	0.1346	0.1319		
$p = 0.75$	0	0	0.0092	0.0068	0.0039	0.2062		

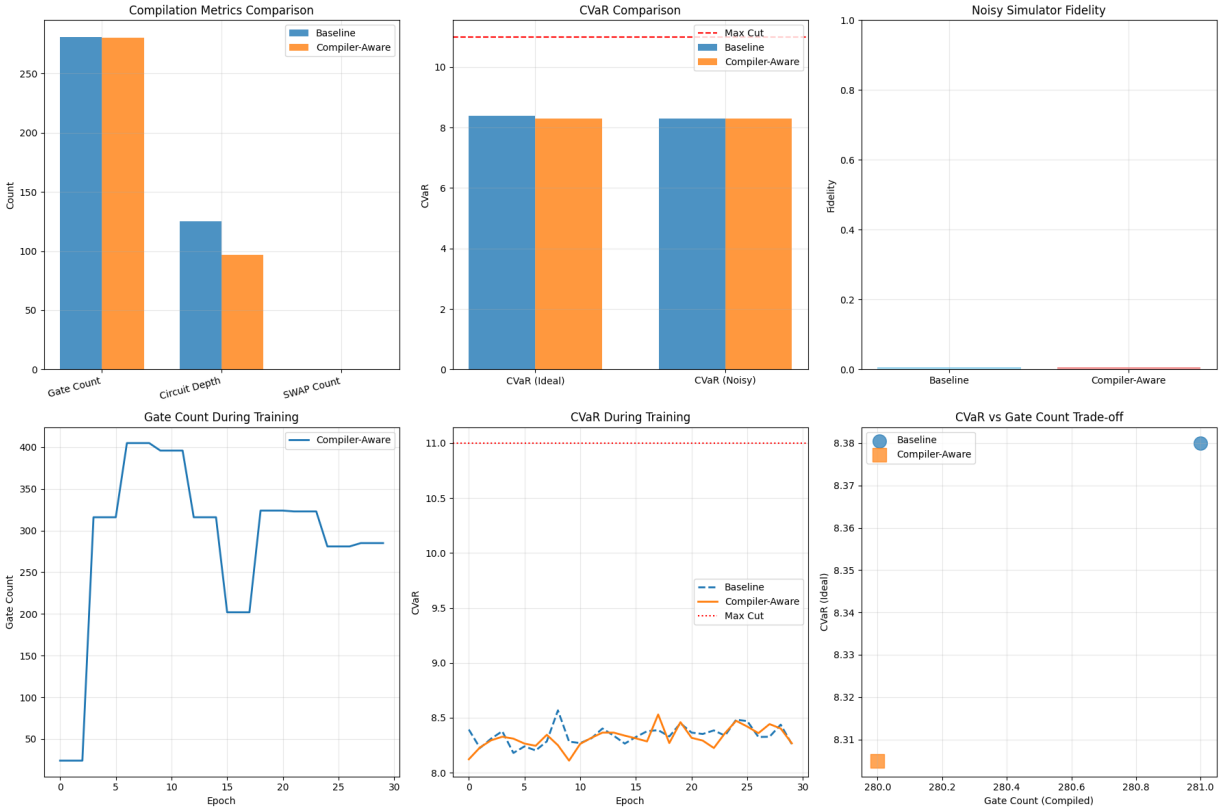

Figure 13: Compiler-aware vs. baseline comparison for an 8-node ER instance with $p = 0.25$.

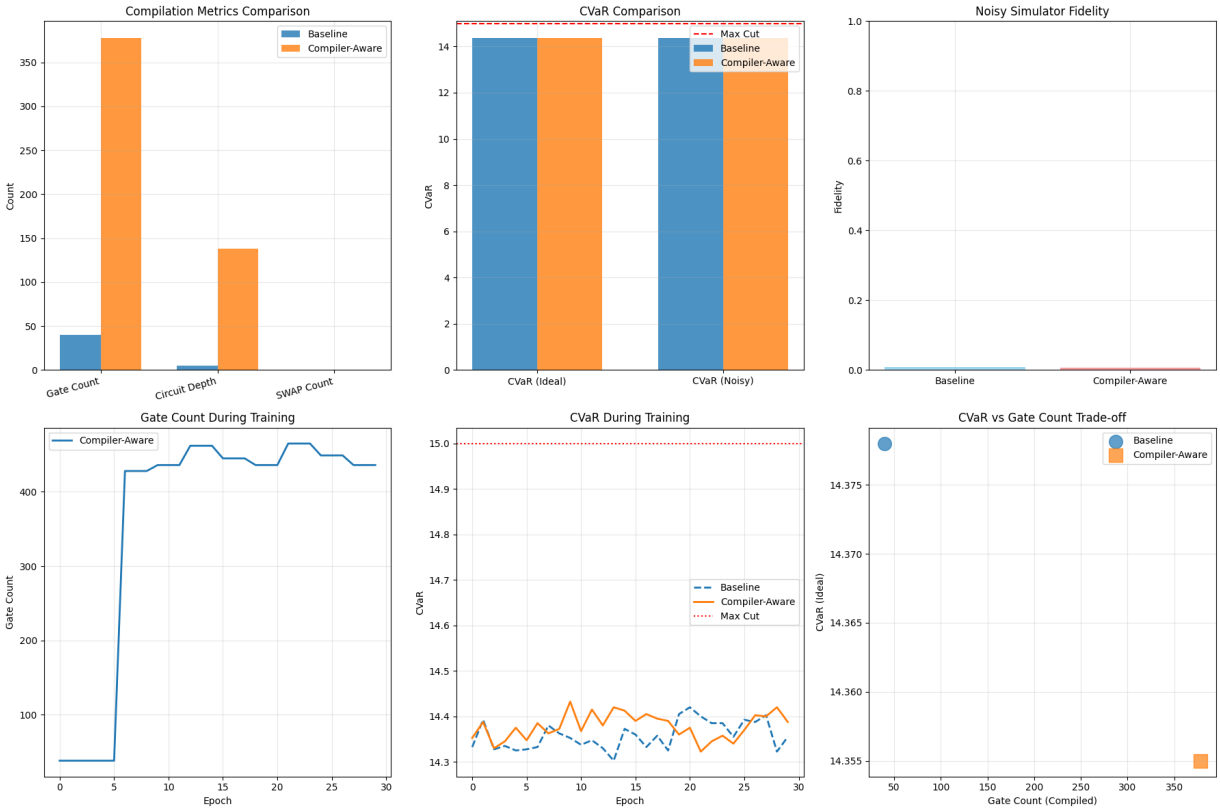

Figure 14: Compiler-aware vs. baseline comparison for an 8-node ER instance with $p = 0.75$.